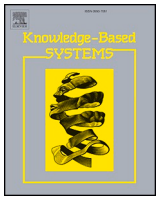
Knowledge-Based Systems 324 (2025) 113833

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys



An intelligent zero-day attack detection system using unsupervised machine learning for enhancing cyber security

Apoorva Jain [a,b,*], Renu Bagoria [c], Praveen Arora [d]

[a] Department of Computer Science & Engineering, Jagannath University, Sitapura, Jaipur, Rajasthan 302022, India
[b] School of Computer Science and Engineering, IILM University, Greater Noida, Uttar Pradesh 201306, India
[c] Department of Computer Science & Engineering,Jagannath University, Sitapura, Jaipur, Rajasthan 302022, India
[d] Jagan Institute of Management Studies, New Delhi, Delhi 110085, India

ARTICLE INFO

Keywords:
Cyber security
Zero-day attacks
Unsupervised Machine Learning
Min-max Normalization
Hippopotamus Optimization Algorithm
Enhanced K-means Clustering and Pufferfish Optimization Algorithm

ABSTRACT

Network intrusion attacks have proliferated globally, particularly the new and unknown attacks referred to as zero-day attacks. Zero-day network intrusion attacks are common cyber security threats that attempt to exploit flaws in a network system. Moreover, unsupervised learning techniques are crucial for detecting zero-day network intrusion attacks due to the drawbacks of existing methods, which include reliance on duplicate features and the inability to obtain labeled data. An Intelligent Zero-day Attack Detection System using Unsupervised Machine Learning (ZdAD-UML) model is introduced in this research study to enhance the cybersecurity of IoT networks. Pre-processing is done first to get the gathered data ready for more analysis. This includes data cleaning, Min-max normalization, and one-hot encoding. To minimize the data redundancy and lessen the computational complexity, the significant features are selected using the Hippopotamus Optimization Algorithm (HOA). The Enhanced K-means Clustering-based Weighted Crayfish Auto Encoder (EKC-WCAE) is used in the feature extraction process to extract the network flow features. Subsequently, the anomaly score is calculated and compared with the threshold value to detect potential zero-day attacks. Furthermore, the threshold calculation is accomplished using the Pufferfish Optimization Algorithm (POA). According to the results, the suggested method uses the IoT-23 and CICIoT2023 datasets to produce accuracy rates of 98.65 % and 98.51 % in the 20–80 Test-Train partition. For 30–70 Test-Train partition, the proposed method attained an accuracy of 98.73 % and 98.93 % using IoT-23 and CICIoT2023 datasets, respectively.

1. Introduction

The Internet has become a vital tool for work, education, and entertainment in today's technologically evolved society. The Internet has become a necessary component of daily life [1]. It is widely acknowledged as one of the key elements of the modern business environment. Attacks are more likely as networks are used more frequently. Every year, network and system security becomes harder to maintain [2]. Real-time assault defense is challenging, and reducing false alarm rates is one of the most important aspects of cyber protection. According to the report, the amount of vulnerabilities discovered each year has been steadily rising, with 233,758 new vulnerabilities discovered in 2023 alone [3]. Using intrusion detection systems (IDS) is an effective way to find cyberattacks on a system. An essential tool for cyber defense, an intrusion detection system (IDS) monitors system anomalies [4].

Using a signature-based methodology, conventional intrusion detection systems (IDS) compare aberrant data to the attack's known signature. This approach is time-consuming, intricate, and prone to overlooking new types of infiltration [5]. Although the signature-based approach has demonstrated acceptable detection rates and accuracy in operational settings, it may miss zero-day attacks due to the high cost of updating the signature library [6,7]. A machine learning-based system that can predict new instances of data by learning from training data is another cutting-edge intrusion detection technology that fails to identify zero-day attacks, which go undiscovered for a long time and cause substantial harm [8]. By examining the network data, anomaly detection is a useful and effective technique to identify any departure from the regular behavior of the network system [9]. The supervised and semi-supervised anomaly detection techniques can be used to train the IDS with known and labelled anomalous and expected data if labelled

* Corresponding author.
E-mail address: apoorva.jnit2008@gmail.com (A. Jain).

https://doi.org/10.1016/j.knosys.2025.113833
Received 23 May 2024; Received in revised form 20 March 2025; Accepted 20 May 2025
Available online 31 May 2025

data for IDS training is available [10–12].

The only remaining technique for detection in the event of zero-day attacks is IDS based on unsupervised training, as the data needed for IDS training is not accessible beforehand. Algorithms for clustering are essential for identifying zero-day threats [13]. The importance of these tools comes from their ability to spot new attack patterns, strange things, and complicated data structures that are part of zero-day attacks when the past data needed to train detection algorithms is not available [14]. In particular, these algorithms excel in spotting anomalies, which are defined as non-standard actions taken by a system or network and may be signs of zero-day attacks. They can adapt to new threats when conventional detection methods fail since they are self-sufficient and do not rely on known attack signals [15].

In addition, these algorithms can handle a range of high-dimensional datasets, which helps them process huge volumes of data in real time. In particular, they are able to capture the complicated and multi-dimensional structure of DDoS attacks inside network traffic [16]. Cyber security professionals can use clustering approaches to better effectively identify and prevent potential zero-day DDoS attacks [17]. Compared to individual clustering algorithms, which perform badly in this area, the ensemble model eliminates the limitations and biases of the individual model and provides a more accurate identification of a zero-day attack [18]. Label collection for zero-day attacks is challenging and nearly impossible due to the supervised algorithms used in earlier research on cyber-attack detection in Internet of Things networks, particularly DDoS operations that consume millions of data [19–21]. Moreover, there is a serious false positive problem with unsupervised intrusion detection systems when trying to detect zero-day cyberattacks [22].

1.1. Motivation and problem statement

Because zero-day network intrusion attacks can exploit undiscovered flaws in a network system, they pose a major danger to cybersecurity. These attacks, sometimes called “new unknown threats,” can cause long-term harm to a network if they are not identified and successfully prevented by thorough network traffic analysis supported by machine learning (ML)-based intrusion detection systems. Because zero-day attacks are invisible and can evade traditional security procedures, they are highly good at infiltrating systems. In addition, before countermeasures can be developed, the rapid exploitation of zero-day vulnerabilities increases the risk of serious and widespread cyberattacks that could cause financial losses, data breaches, and reputational damage. Zero-day attacks can be challenging to detect since they can avoid traditional detection techniques and lack any established patterns that conventional security procedures can employ to identify these attacks. This unpredictable nature emphasizes how important defense strategies and continuous network traffic monitoring are to preventing the attacks these sophisticated cyber threats offer, particularly advanced persistent threats often connected to zero-day attacks. Furthermore, the shortcomings of existing techniques, such as their dependence on duplicated characteristics and difficulties in getting labeled data, highlight the necessity of using unsupervised learning techniques to identify zero-day network intrusion attacks. Unsupervised learning techniques are important for effective intrusion detection in contemporary cyber security environments because of the dynamic and growing nature of zero-day threats. Thus, in the proposed research, an innovative machine learning model for intrusion detection can be built using unsupervised machine learning. The major objectives of the proposed work for zero-day attack detection are presented below as follows:

- ➢ To propose a novel Intelligent Zero-day Attack Detection System using Unsupervised Machine Learning for enhancing cyber security in IoT networks.
- ➢ To select the optimal features for minimizing data redundancy and reducing computational complexity to enhance the detection performance.
- ➢ To employ an enhanced unsupervised machine learning model to extract network flow features and group similar traffic patterns into clusters.
- ➢ To offer a bio-inspired meta-heuristic algorithm that mimics the natural characteristics of pufferfish for threshold calculation.
- ➢ To compare the classification performance against existing methods for determining the superiority in detection.

The remaining portion of the paper is structured as follows: The prior relevant works on Zero-day Attack Detection are covered in Section 2. The specifics of the proposed methodology applied in this investigation are detailed in Section 3. The performance outcomes attained by the proposed method are shown in Section 4. Section 5 offers recommendations for further research and concludes the study.

2. Related works

Some recent works on zero-day attack detection are analyzed in this section.

Sakthi Murugan et al. [23] have devised a machine learning methodology to evaluate zero-day vulnerabilities. This approach employed the software systems to detect zero-day vulnerabilities. The approach combines an auto-encoder model and a Deep Learning (DL) model to detect abnormal data patterns. Additionally, this study presents a comparison between the autoencoder model and the single class-based Support Vector Machine (SVM) strategy for identifying outliers. The CICIDS2017 dataset is utilized to gather input data. The primary obstacles encountered in the procedure are the elevated time complexity and the subpar performance rate.

To detect zero-day distributed denial-of-service attacks on Internet of Things networks, Roopak et al. [24] introduced an unsupervised method. The dimensionality of the data from the network is reduced throughout the feature selection process. For unsupervised data categorization utilizing a hard voting technique into attack and normal categories, the ensemble model consists of one-class SVM, GMM, and K-means. In an extensive review, the CIC-DDoS2019 datasets are utilized. The combination of high storage capacity and subpar performance makes the process more arduous.

In order to detect potential cyber threats, Omer et al. [25] used improved frameworks for machine learning. The proposed method for cyberattack detection uses particle swarm optimization (PSO) and support vector machine (SVM) classifiers. The KDD-CUP 99 dataset was used to confirm that the recommended method worked as intended. Low security and a lengthy computational process are the study’s two main limitations.

In order to protect against zero-day attacks, Ekong et al. [26] introduced a machine learning method for classification and the viewpoints of entities about its effects. On malware zero data, exploratory data analysis (EDA) was carried out. Principal Component Analysis (PCA) was utilized for feature selection to identify the most crucial properties in the dataset, and the Random Forest (RF) Algorithm was used for classifying zero-day attacks. The approach is limited by two key challenges: a low-performance rate and a large computational complexity.

In order to identify zero-day network intrusions, Alam et al. [27] used a machine learning approach. This approach integrates Nevergrad, the BAT algorithm, and the anomaly-based extended isolation forest algorithm. Further, the model was evaluated using data from 5 G networks. The second part of the process involves training the model with a subset of features and then adjusting the hyperparameters using the Nevergrad optimizer. Difficulty handling data, slow convergence speed, and restricted exploration are some of the method’s shortcomings.

Traditional IDS methods often struggle with the high dimensionality

and dynamic nature of IoT data, leading to inefficiencies in threat detection and response. To solve this, Abbas et al. [28] integrated decision trees with Bayesian hyperparameter optimization to enhance IDS performance in complex IoT environments. Utilizing the comprehensive IoT-23 dataset, methodology not only demonstrated a significant improvement in detecting diverse attack scenarios but also achieved a remarkable accuracy rate of 96.07 % and a weighted average F1 score of 96.03 %. However, Bayesian hyperparameter optimization might not provide as much insight into the specific relationships between hyperparameters.

He et al. [29] used Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) networks for intrusion detection. This article optimizes LSTM to obtain a GRU (Gate Recurrent Unit) network and compares it with traditional logistic regression (Softmax) classification methods. The research results of this article show that using dropout to train neural networks can weaken the interaction between neurons and effectively avoid overfitting. The accuracy of GRU-LSTM and GRU-Softmax in the IoT-23 (Internet of Things) database is 96 % and 76 %, respectively. This article proposes an IoT data intrusion detection method based on the GRU-LSTM algorithm, which has more advantages. This study didn't pre-process the input data, which led to less accuracy and high error results.

The previous works suffered from overfitting and non-optimized intrusion detection. To tackle this, Kumar et al. [30] introduced data fusion is effectuated by the horizontal emergence of two various datasets such as IoT-23 and IoTID20. The fused dataset is forwarded to the proposed support vector machine (SVM) based Binary Grasshopper Optimization algorithm (BGO). The proposed BGO-SVM detects the intrusion in the fused dataset as normal and abnormal. The proposed method attains the optimal values of accuracy, detection rate, F-measure, and recall as 97 %, 99 %, and 96 % apart. SVMs can be sensitive to noisy data or outliers.

Hizal et al. [31] presented deep learning architecture for detecting DDoS attacks in IoT networks. First of all, pre-processing operations were carried out on the CICIoT2023 dataset, and three different datasets were obtained to perform different classification processes. This paper trained fully connected, convolutional, and LSTM-based deep learning models for detecting DDoS attacks and sub-classes. According to the results on a partially balanced sub-dataset, two staged models performed better than baseline models such as Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), Recurrent Neural Networks (RNN). According to the test results, the proposed CNN-based two-stage model has an accuracy of 91.27 %. The absence of parameter optimization leads to more training of the model.

Gulzar et al. [32] introduced a deepCLG hybrid learning model designed to improve network intrusion detection systems. Initially, the dataset is pre-processed and then fed into deepCLG, which is the combination of convolutional neural network (CNN) and long short-term memory (LSTM), and the gated recurrent unit (GRU) with the capsule network (CN). The efficiency of the proposed method is evaluated by using CICIoT 2023 and UNSW_NB15 datasets. The deepCLG method attains an overall accuracy of 98 % in intrusion detection. A constraint of this approach is the complexity associated with parameter selection during the training phase.

Alzahrani et al. [33] introduced a hybrid DL model that combines CNN with LSTM for energy-aware, anomaly-based IDS. CICIoT2023, KDD-99 and NSL-KDD datasets will be used to evaluate the performance of the suggested IDS model based on key metrics, including latency, energy consumption, false alarm rate, and detection rate metrics. Experimental findings show that the suggested method obtained an accuracy rate of over 92 % and a false alarm rate below 0.38 %. The complexity and size of a dataset can significantly impact the time and computational resources needed for feature selection algorithms.

Gueriani et al. [34] combined CNN with LSTM, which facilitates the detection and classification of IoT traffic into binary categories. Increased accuracy and efficiency can be attained by utilizing CNN's spatial feature extraction capabilities for pattern recognition and LSTM's sequential memory retention for recognizing complex temporal relationships. The New CICIoT2023 dataset was used for both training and final testing while further validating the model's performance through a conclusive testing phase utilizing the CICIDS2017 dataset. The suggested approach produces an accuracy rate of 98.42 %, accompanied by a minimal loss of 0.0275. Dataset complexity and size are key factors that influence the time and computational resources required by feature selection algorithms. Table 1 shows the analysis of the existing work on zero-day attack detection.

The review of previous research reveals a number of drawbacks, including slow convergence speed, large computational complexity, and a lengthy time need, among others. This paper introduces a novel ZdAD-UML strategy to address these challenges and increase performance.

3. Proposed methodology

To improve cyber security in IoT networks, this effort intends to offer a ZdAD-UML model. The suggested approach's architecture is depicted in Fig. 1.

The acquired data is initially pre-processed with one-hot encoding and min-max normalization to prepare it for additional analysis. In order to get the gathered data ready for analysis, it is first pre-processed using Min-max Normalization and One-hot encoding. Feature selection is conducted utilizing HOA to pick the most relevant features and reduce model complexity. EKC-WCAE extracts network flow features to find anomaly scores and enhance cybersecurity. Furthermore, the latent representations are clustered using the K-means clustering technique after training the autoencoder (AE) to arrange identical traffic patterns into clusters. The reconstruction error is calculated using the initial and rebuilt data. To identify possible zero-day attacks, the anomaly score is computed and compared to the threshold value. The suggested method determines the cutoff value by utilizing POA. As a result, the proposed EKC-WCAE-based unsupervised modeling that incorporates feature learning abilities of AE with clustering abilities of K-means offers superior performance to detect and mitigate zero-day DDoS attacks in IoT networks while also improving resilience and security against emerging threads.

3.1. Data collection

The proposed approach uses IoT-23 and CICIoT2023 datasets for detecting zero-day attacks. The details about these datasets are given below,

3.1.1. IoT-23 dataset

Using the IoTID20 [35] dataset, several studies have built detection methods for zero-day attacks. The IoTID20 dataset is utilized for both binary and multiple-class classification tasks. The IoT-23 dataset contains twenty-three scenarios or captures of different types of IoT network traffic. It includes twenty malware captures executed in IoT devices in addition to three grabs for benign IoT device traffic. https://www.stratosphereips.org/datasets-iot23. The IoT-23 dataset has two main classes: Malicious and Benign.

3.1.2. CICIoT2023 dataset

To make it easier to construct security analytics software for the Internet of Things, the CICIoT2023 [36] Dataset was created. There are seven sections in this dataset: DDoS, Recon, Web-based, Brute Force, Spoofing, and Mirai. There were 105 Internet of Things devices targeted in 33 separate attacks. When these attacks occur, it's because individual IoT devices are going after other IoT devices. https://www.kaggle.com/datasets/akashdogra/cic-iot-2023. The CICIoT2023 dataset provides labels for network traffic, classifying each record into one of the following categories:

Table 1
Existing works analysis on zero-day attack detection.

Author name	Technique used	Dataset used	Limitation
SakthiMurugan et al. [23]	SVM, DL	CICIDS2017	The higher time complexity and low-performance rate are the major challenges presented in the method.
Roopak et al. [24]	SVM, GMM, and K-means	CIC-DDoS2019	The high storage space and low performance make the method more challenging.
Omer et al. [25]	PSO + SVM	KDD-CUP 99	The limitations, such as low security and high time required for the computational process
Ekong et al. [26]	EDA, PDA, and RF	Zero-day malware attack dataset	The low-performance rate and high computational complexity are the major challenges presented in the method.
Alam et al. [27]	BAT algorithm	5 G NIDD	low convergence speed, limited exploration
Abbas et al. [28]	Bayesian hyperparameter optimization	IoT-23 dataset	Bayesian hyperparameter optimization might not provide as much insight into the specific relationships between hyperparameters.
He et al. [29]	GRU-LSTM and GRU-Softmax	IoT-23 dataset	They didn't pre-process the input data, which led to lower accuracy and high error results.
Kumar et al. [30]	BGO-SVM	IoT-23 and IoTID20	SVMs can be sensitive to noisy data or outliers.
Hizal et al. [31]	CNN, DNN, RNN, LSTM	CICIoT2023	The absence of parameter optimization leads to more training of the model.
Gulzar et al. [32]	deepCLG	CICIoT 2023 and UNSW_NB15	A constraint of this approach is the complexity associated with parameter selection during the training phase.
Alzahrani et al. [33]	CNN with LSTM	CICIoT2023, KDD-99 and NSL-KDD	The complexity and size of a dataset can significantly impact the time and computational resources needed for feature selection algorithms.
Gueriani et al. [34]	CNN with LSTM	CICIoT2023 and CICIoT2017	Dataset complexity and size are key factors that influence the time and computational resources required by feature selection algorithms.

- **DDoS:** Distributed Denial of Service attacks aiming to overwhelm a target system with traffic.
- **DoS:** Denial of Service attacks, typically originating from a single source.
- **Recon:** Reconnaissance attacks are designed to gather information about the target system.
- **Web-Based:** Attacks targeting web applications, such as SQL injection and cross-site scripting.
- **Brute Force:** Attempts to guess passwords or credentials through repeated trials.
- **Spoofing:** Attacks that attempt to impersonate legitimate entities.
- **Mirai:** Attacks leveraging the Mirai botnet, known for targeting IoT devices.
- **Benign:** Normal, legitimate network traffic from uninfected devices.

3.2. Pre-processing

Three stages are used to pre-process input data, which are discussed below.

3.2.1. Data cleaning

Data cleansing is a stage in the pre-processing of data. The main purposes of data cleaning are noise reduction and data loss [37]. Two commonly used data cleaning strategies are Ignore Missing (IM) and Missing Common (MC). Using the IM approach, records with at least one attribute loss are eliminated. MC, or imputation, differs from IM in that it substitutes missing data for the most frequent data for a given attribute.

3.2.2. Min-max normalization

The most basic normalizing method is min-max [38], which produces the usual numerical range of scores between 0 and 1. It is stated as,

$$a' = \frac{(a - \min(A))}{\max(A) - \min(A)} \tag{1}$$

where, the normalized score of $a$is denoted asa', $\max(A)$represents the maximum value and $\min(A)$is denoted as the minimum value.

3.2.3. One-hot encoding

It is common practice to employ one-hot encoding [39] when label encoding is inadequate and there is no ordinal link between two nominal categorical variables for a categorical random variable $A$with Kdissimilar values$a_1, a_2, \ldots\ldots, a_k$. Every component of a vector b that represents a specific value $a_i$is zero, with the exception of the i^{th}component, which is encoded as 1.

Suppose A takes values from the set $W = e, f, h,$ and$a_1 = e, a_2 = f, a_3 = h$. A one–hot encoding for $a$is$(1,0,0), (0,1,0)$ and$(0,0,1)$.

3.3. Feature selection

Utilizing HOA [40], the best features for the suggested strategy are chosen in order to reduce feature dimensionality problems and model complexity. The optimization algorithm uses its behaviors and strategies to identify an ideal solution through a series of iterations based on the collected pre-processed data, which generates an effective population. There are two primary stages to the feature selection model's deployment: exploration and exploitation. Hippopotamuses are potential optimization problem solutions in the HOA algorithm; hence, the values of the decision variables are represented by the position updates of each hippopotamus in the search space. The details about HOA are described below,

3.3.1. Hippopotamus optimization algorithm

The HOA is a population-based optimization system that makes use of hippopotamus search agents. In HOA, every hippo is represented as a vector, and a matrix is used to classify the hippo population quantitatively. Similar to the conventional optimization technique, HOA generates randomized initial solutions as part of its initialization procedure. The vector of decision variables that are produced in this stage is expressed as,

$$A_x : a_{xy} = l_y + r.(u_y - l_y), x = 1, 2, 3\ldots\ldots, N, y = 1, 2, 3, \ldots\ldots\ldots m \tag{2}$$

where $a_x$denoted as the location of x^{th}candidate solution, $r$denoted as an arbitrary amount among 0 and 1, $l$and $b$are lower and upper bounds of y^{th}decision variable, individually. Nrepresents population size for hippopotamus within the herd, m referred to as a few problem-specific decision variables. The population matrix is expressed as,

$$A = \begin{bmatrix} A_1 \\ \vdots \\ A_x \\ \vdots \\ AN \end{bmatrix}_{N\times m} = \begin{bmatrix} a_{1,1} & \cdots & v_{1,y} & \cdots & a_{1,m} \\ \vdots & \ddots & \vdots & \iddots & \vdots \\ a_{x,1} & \cdots & a_{x,y} & \cdots & a_{x,m} \\ \vdots & \iddots & \vdots & \ddots & \vdots \\ a_{N,1} & \cdots & a_{N,y} & \cdots & a_{N,m} \end{bmatrix}_{N\times m} \tag{3}$$

3.3.1.1. Exploration (position update of hippopotamus in pond or river). The mathematical representation of the position of male hippopotamus members in a lake or pond is as follows,

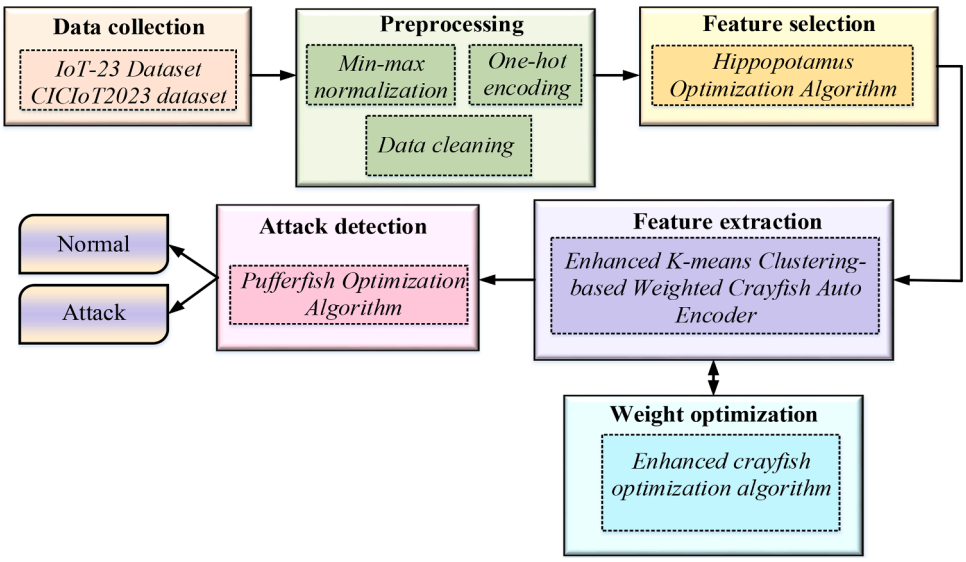


Fig. 1. Architecture of proposed approach.

$$a_x^{m.hipp} : a_{xy}^{m.hipp} = a_{xy} + b1.\left(D.hipp - T_1 a_{xy}\right) \tag{4}$$

for $x = 1, 2, \ldots., \left[\frac{N}{2}\right] and y = 1, 2, \ldots, m$ where $a_x^{m.hipp}$ denotes the position of the male hippopotamus, $D.hipp$ represents the position of the dominant hippopotamus, $\vec{r}_{1,\ldots\ldots,4}$ denotes random vector among 0 and 1. T_1 and T_2 is an integer among 1 and 2. mg_x represents the average values of a randomly selected hippopotamus, with each hippopotamus(A_x) having an equal chance of being included in the calculation and $b1$ denotes random number among 0 and 1. Integers q_1 and q_2 can take on values between 1 and 0.

$$h = \begin{cases} T_2 \times \vec{r} + (\sim q_1) \\ \quad 2 \times \vec{r}_2 - 1 \\ \quad\quad \vec{r}_3 \\ T_1 \times \vec{r}_4 + (\sim q_2) \\ \quad\quad r_5 \end{cases} \tag{5}$$

$$S = \exp\left(-\frac{s}{S}\right) \tag{6}$$

$$A_x^{fb.hipp} : a_{xy}^{fb.hipp} \begin{cases} a_{xy} + h_1.(D.hippo - .T_2 mg_x) S > 0.6 \\ [T] else \end{cases} \tag{7}$$

$$[T] = \begin{cases} a_{xy} + h_2.(mg_x - D.hippo) r_6 > 0.5 \\ l_y + r_7.(u_y - l_y) else \end{cases} \tag{8}$$

for $x = 1, 2, \ldots\ldots\ldots\ldots, \left[\frac{N}{2}\right] and y = 1, 2, \ldots, m$

The position of the female or immature hippopotamus is denoted as $\left(A_x^{fb.hipp}\right)$ within the herd, which is expressed in Eqs. (7) and (8). If S is superior to 0.6, it represents an immature hippopotamus away from its mother. If r_6 is a positive integer between 0 and 1, then the young hippopotamus is either still with the herd or quite near to it, even if it has separated from its mother. Otherwise, it is no longer part of the herd. h_1 and h_2 are the numbers or vectors randomly chosen from five scenarios in h . Eqs. (9) and (10) denote the position of man and feminine or immature hippopotamus within the herd. fit denotes objective function value.

$$A_x = \begin{cases} A_x^{m.hipp} f_x^{m.hipp} < fit \\ \quad A_x else \end{cases} \tag{9}$$

$$A_x = \begin{cases} A_x^{fb.hipp} fit_x^{fb.hipp} < fit \\ \quad A_x else \end{cases} \tag{10}$$

Using h vectors, T_1 and T_2 scenarios improves global search and exploration in the proposed HOA. It leads to an enhanced global search and enhances the proposed algorithm's exploration process.

3.3.1.2. Exploration (hippopotamus defence against predators)

The position of the predator in the search space is expressed as,

$$P : P_y = l_y + \vec{r}_8.\left(u_y - l_y\right), y = 1, 2, 3, \ldots m \tag{11}$$

where $\vec{r}_8$ denotes a random vector among 0 and 1.

$$\vec{D} = \left|\text{Predator}_y - a_{xy}\right| \tag{12}$$

Eq. (12) implies the far of x^{th} hippopotamus to a predator. At this time, the hippopotamus agrees on defensive behavior based on factors fit_{Predator} for protecting itself against predators. If fit_{Predator} is less than fit_x denotes the predator is near to the hippopotamus. If fit_{Predator} is greater, it signifies that the hippopotamus's territory is closer to a predator or intruding object Eq. (13).

$$A_x^{hippR} : a_{xy}^{hippR} = \begin{cases} R\vec{L} \oplus \text{Predator}_y + \left(\frac{f}{(\sigma d \times \cos(2\pi g))}\right).\left(\frac{1}{\vec{D}}\right) fit_{\text{Predator}_y} < fit_x \\ R\vec{L} \oplus \text{Predator}_y + \left(\frac{f}{(\sigma d \times \cos(2\pi g))}\right).\left(\frac{1}{2 \times \vec{D} + \vec{r}_9.}\right) fit_{\text{Predator}_y} \geq fit_x \end{cases} \tag{13}$$

for $x = \left(\frac{N}{2}\right)+1, \left(\frac{N}{2}\right)+2, \ldots\ldots\ldots, Nandy = 1, 2 \ldots, m$where A_x^{hippR}denotes the position of the hippopotamus, which was faced with predators. In a hippopotamus assault, the predator's position can change suddenly; hence, the symbol $R\overrightarrow{L}$ is used to represent an arbitrary vector with levy distribution. $f$denotes uniform, arbitrary amount among 2 and 4, Dmeans uniform, arbitrary amount among -1 and 1. The mathematical expression for Levy movement's random movement is,

$$Levy(v) = 0.05 \times \frac{\omega \times \sigma_\omega}{|v|^{\frac{1}{\vartheta}}} \tag{14}$$

$$\sigma_\omega = \left[\frac{\Gamma(1+\vartheta)\sin\left(\frac{\pi\vartheta}{2}\right)}{\Gamma\left(\frac{(1+\vartheta)}{2}\right)\vartheta 2^{\frac{(\vartheta-1)}{2}}}\right]^{\frac{1}{\vartheta}} \tag{15}$$

where $\omega$and $v$are random numbers in $[0, 1]$, correspondingly. ϑrepresents constant ($\vartheta = 1.5$), Γdefines an abbreviation for the Gamma function, and $\sigma_\omega$is obtained from Eq. (15).

$$A_x = \begin{cases} A_x^{hippR} fit_x^{hippR} < fit_x \\ A_x fit_x^{hippR} \geq fit_x \end{cases} \tag{16}$$

3.3.1.3. Exploitation (Hippopotamus escaping from the predator)

To imitate this phenomenon, a random spot is constructed near the hippopotamus locations.

$$l_y^{local} = \frac{l_y}{s}, u_y^{local} = \frac{u_y}{s}, s = 1, 2, \ldots, S \tag{17}$$

$$A_x^{hippΣ} : a_{xy}^{hippΣ} = a_{xy} + r_{10} \cdot \left(l_y^{local} + z_1 \cdot \left(u_v^{local} - l_y^{local}\right)\right) \tag{18}$$

for $x = 1, 2, 3, \ldots.N$and $y = 1, 2, \ldots., m$here s stands for current iteration and Sdenotes maximum iteration. $A_x^{hippΣ}$denotes the location of the hippopotamus, which was examined to discover the nearest, not dangerous, location. z_1 represents a number or arbitrary vector that is carefully chosen at random from three possible possibilities z. The mathematical expression $z$is given below,

$$z = \begin{cases} 2 \times \overrightarrow{r}_{11} - 1 \\ r_{12} \\ r_{13} \end{cases} \tag{19}$$

$$A_x = \begin{cases} A_x^{hippΣ} fit_x^{hippΣ} < fit_x \\ A_x fit_x^{hippΣ} \geq fit_x \end{cases} \tag{20}$$

The Pseudo code for the proposed HOA is described in Table 2.

Consequently, the proposed approach chooses the best features for zero-day attack detection by achieving the best fitness value. This tends to reduce feature dimensionality issues and overall processing time. By using this optimization the, features like 'flow_duration', 'Header_Length', 'Duration', 'Rate', 'Srate', 'Drate', 'fin_flag_number', 'cwr_flag_number', 'ack_count', 'fin_count', 'urg_count', 'HTTPS', 'DNS', 'SMTP', 'SSH', 'IRC', 'UDP', 'DHCP', 'IPv', 'Tot sum', 'Min', 'Std', 'Tot size', 'Magnitue', 'Radius', and 'Variance' are selected from the IoT-23 dataset. For the CICIoT2023 dataset, IP addresses associated with port 1 are selected by this optimization algorithm.

Table 2
Pseudocode for proposed HOA.

Pseudo code for proposed HOA
Start HOA
Describe an optimization problem.
Set maximum iterations (S)number and the number of hippopotamus (N)
Using Eq. (2) and fitness function evaluation for the starting population, determine the positions of all the hippopotamus
For $s : 1 : S$
Revise the dominant hippocampus's location according to the value criterion of the goal function.
Phase 1: Update the hippopotamus's location in the river or pond **(Exploration phase)**
For $x = 1 : N/2$
Compute x^{th}hippopotamus new position using Eqs. (4), (7)
Update the position of x^{th}hippopotamus using Eqs. (9), (10)
End for
Phase 2: hippopotamus defense against predators **(Exploration phase)**
For $x = 1 : N/2 : N$
Produce the predator's random position using Eq. (11)
Compute x^{th}hippopotamus new position using Eq. (13)
Update the position of x^{th}hippopotamus using Eq. (16)
End for
Phase 3: hippopotamus escaping from predator (**Exploitation phase**)
Estimate new bounds of variable decision using Eq. (17)
For $x = 1 : N$
Compute the new position for x^{th}hippopotamus using Eq. (18)
Update the location of x^{th}hippopotamus using Eq. (16)
End for
Save the finest applicant solution obtained so far
End for
Output of the finest solution of fitness function attained by proposed HOA
End HOA

3.4. Feature extraction

The EKC-WCAE model trains selected features to distinguish zero-day attack detection. The details about the EKC-WCAE model are described below,

3.4.1. Enhanced K-means clustering-based weighted crayfish auto encoder

Initially, the selected features are clustered using a k-means clustering algorithm. The steps of the k-means algorithm [41] are designated as below,

Step 1: Randomly select optimal$K$features from the HOA algorithm as the initial cluster center:

$$V_y(X), y = 1, 2, 3, .., K \tag{21}$$

Step 2: Get distance among all elements and $V_y(X)$in the cluster:

$$D\left(a_i, V_y(X)\right), x = 1, 2, 3., n; y = 1, 2, \ldots.K \tag{22}$$

After the feature will be allocated to the nearest cluster, if satisfied

$$D\left(a_i, V_y(X)\right) = \min\left\{D\left(a_i, V_y(X)\right)\right\} \tag{23}$$

Then $a_x \in C_k$

Step 3: Error square sum criterion function can be attained:

$$Y_c = \sum_{y=1}^{c} \|a_k^x - V_y(X)\|^2 \tag{24}$$

Step 4: If $|Y_c(X) - Y_c(X-1)| < \xi$, stop and output the cluster result. Otherwise, continue to iterate, compute the clustering center $V_y(X) = \frac{1}{n}\sum_{y=1}^{n_y} a_k^y$, and go to step 2 until $|Y_c(X) - Y_c(X-1)| < \xi$. The clustered features are trained by the AE model, and the details about the AE model are described below,

An input layer, a hidden layer, and an output layer comprise the AE neural network's three-layer architecture, which is unsupervised [42].

Encoding and decoding are the two essential phases involved in the functioning of an automatic encoder. Each step has a specific definition: The process of encoding information from the input layer to the hidden layer.

$$N = h_{\theta 1}(I)\sigma\left(K_{xy}I + \psi 1\right) \tag{25}$$

The process of decoding from the hidden layer to the reconstruction layer.

$$F = h_{\theta 2}(E)\sigma\left(K_{yz}E + \psi 2\right) \tag{26}$$

where, $I = (i_1, i_2, \ldots\ldots i_n)$ - input data vector, $F = \left(f_1, f_2, \ldots\ldots f_n\right)$ -reconstruction vector of input data, $E = (e_1, e_2, \ldots\ldots e_n)$ -low dimensional vector output from the hidden layer. Hidden layer neurons' and output layer neurons' activation functions$h_{\theta 1}(\oplus)$and $h_{\theta 2}(\oplus)$, respectively, serve to map network summation results to [0, 1]. The sigmoid function is employed as the activation function in this paper

$$h_{\theta 1}(\oplus) - h_{\theta 2}(\oplus) = \frac{1}{1 + d^{-x}} \tag{27}$$

Calculate reconstruction errors where the squared-error function is used $AB(W, \psi)$ between N and F, where N defines a number of input samples.

$$AB(W, \psi) = \frac{1}{2N}\sum_{r=1}^{N} \|F^{(r)} - I^{(r)}\|^2 \tag{28}$$

The ECOA approach is employed to refine the hyperparameters and optimize the weights of the AE model. The ECOA technique is used in combination with the proposed residual depthwise separable convolution network model to improve performance and fine-tune hyperparameters. Furthermore, optimization techniques are utilized to improve the prediction process, resulting in enhanced accuracy and performance simultaneously. The ECOA algorithm proposes the attributes of puffer fish, such as their food search and hunting behavior. These strategies are used to modify the corresponding hyperparameters, the external configuration parameters utilized in the DL model, and machine learning training. A hyperparameter configuration that produces accurate prediction results after several iterations is considered the optimal hyperparameter. The following information provides a description of the original Crayfish Optimization Algorithm (COA) [43].

3.4.1.1. Population initialization. Each crayfish is a matrix of $1 \times dim$in a multi-dimensional problem. A problem's solution is represented by each column matrix. In a set of variables $\left\{C_{x,1}, C_{x,2}, C_{x,3} \ldots\ldots C_{x,dim}\{\}\right\}$each variable C_x must lie among upper and lower limits. The purpose of initializing ECOA is to stochastically produce a set of candidate solutions$C$ inside the given space. The candidate solution C according to dimension $dim$and population size$N$. The initialization of ECOA is expressed as,

$$C = [C_1, C_2, \ldots, C_N] = \begin{bmatrix} C_{1,1} & \cdots & C_{1,y} & \cdots & C_{1,dim} \\ \vdots & \cdots & \vdots & \cdots & \vdots \\ C_{i,y} & \cdots & C_{x,y} & \cdots & C_{x,dim} \\ \vdots & \cdots & \vdots & \cdots & \vdots \\ C_{N,1} & \cdots & C_{N,y} & \cdots & C_{N,dim}[] \end{bmatrix} \tag{29}$$

where Cdefines initial population, Ndenotes the number of population, dimdefines population matrix, $C_{x,y}$represents the position of individual x in y^{th}dimension, and $C_{x,y}$value is expressed as,

$$C_{x,y} = lbd_y + \left(ubd_y - lbd_y\right) \times rand \tag{30}$$

where lbd_ydenotes lower bound, ubd_ydenotes upper bound in y^{th}dimension, and $rand$defines random number.

3.4.1.2. Describe the temperature and consumption of crayfish. Crayfish will undergo a different stage of behaviour due to temperature changes. Temperature is expressed as,

$$temp = rand \times 15 + 20 \tag{31}$$

where $temp$ stands for the crayfish's location temperature. The mathematical representation of crayfish is given as,

$$\rho = T_1 \times \left(\frac{1}{\sqrt{2 \times \pi} \times \sigma} \times \exp\left(-\frac{(temp - \mu)^2}{2\sigma^2}\right)\right) \tag{32}$$

where μ specifies the optimal temperature for crayfish, $\sigma$and T_1 are utilized to regulate the consumption of crayfish at different temperatures.

3.4.1.3. Summer resort stage (exploration). When$temp > 30$, the temperature is too high. Crayfish will now opt to spend their summer vacation within the confines of the cave. The cave $C_{shade}$is expressed as,

$$C_{shade} = (C_G + C_L)/2 \tag{33}$$

where $C_G$and C_L are defined as the present population's optimal position and the optimal position reached thus far based on a number of iterations. When $rand < 0.5$it means the lack of competition from other crawfish for caves, crayfish will spend their summer vacation in the cave.

$$C_{x,y}^{i+1} = C_{x,y}^{i} + T_2 \times rand \times \left(C_{shade} - C_{x,y}^{i}\right) \tag{34}$$

where $i$denotes the present repetition amount, and $i + 1$denotes the next-generation repetition amount, and T_2means decreasing curve it is expressed as,

$$T_2 = 2 - (i / I) \tag{35}$$

where I denotes the maximum iteration number.

3.4.1.4. Competition stage (exploitation). Crayfish competes for cave through Eq. (36)

$$C_{x,y}^{i+1} = C_{x,y}^{i} - C_{z,y}^{i} + C_{shade} \tag{36}$$

where $z$denoted as a random individual of crayfish, it is expressed as,

$$z = round(rand \times (N - 1)) + 1 \tag{37}$$

3.4.1.5. Foraging stage (exploitation). The food location is expressed as,

$$C_{food} = C_G \tag{38}$$

Food size $S$is expressed as,

$$S = T_3 \times rand \times \left(fitness_x \,/\, fitness_{food}\right) \tag{39}$$

where T_3defines food factor, signifying biggest food and value 3 is persistent, $fitness_x$denotes x^{th}crayfish fitness value, and $fitness_{food}$denotes fitness value for food. When$S > (T_3 + 1)/2$, it means the size of food is too big. At this point, the crayfish will use its first claw foot to tear food. Mathematically, it is expressed as,

$$C_{food} = \exp\left(-\frac{1}{S}\right) \times C_{food} \tag{40}$$

Foraging behaviour is mathematically expressed as,

$$C_{x,y}^{i+1} = C_{x,y}^{i} + C_{food} \times p \times (\cos(2 \times \pi \times rand)) - \sin(2 \times \pi \times rand) \tag{41}$$

When$S \le (T_3 + 1)/2$, a crayfish desires to change toward food, it eats directly. It is expressed as,

$$C_{x,y}^{i+1} = \left(C_{x,y}^{i} - C_{food}\right) \times p + p \times rand \times C_{x,y}^{i} \tag{42}$$

3.4.1.6. ECOA. Tent chaotic maps are used to improve the COA method. In order to enhance the performance of COA, the original COA's random number is substituted with the Tent chaotic *Tc* map. ECOA is stated as follows,

$$C_{x,y} = lbd_y + \left(ubd_y - lbd_y\right) \times Tc \tag{43}$$

$$Tc^{p+1} = \begin{cases} \dfrac{Tc^p}{\nu}, TC^p \in 0, \nu) \\ \dfrac{(1^\nu - Tc^p)}{(1-\nu)}, TC^p \in [\nu, 1] \end{cases} \tag{44}$$

where p is represented as the present amount of iterations in $a \in (0,1)$ then the$Tc^p \in [0,1]$. To enhance performance, the Tent chaotic map is used. ECOA will find the best solution using the tent chaotic maps, producing good performance and a fast rate of convergence. During testing, EKC-WCAE learns typical network patterns, collects pertinent characteristics from network flows, and detects anomalies.

3.5. Attack detection

An anomaly score is calculated and compared to a threshold (determined using POA) to detect potential zero-day attacks.

3.5.1. Pufferfish optimization algorithm

POA [44] is used to select the optimal threshold value for detecting zero-day attacks. By producing the best and finest value, the POA chooses the optimal threshold value. The details about POA are described below,

3.5.1.1. Initialization. Population initialization for the PO algorithm is mathematically expressed as,

$$P = \begin{bmatrix} P_1 \\ \vdots \\ P_i \\ \vdots \\ P_N \end{bmatrix}_{N \times m} = \begin{bmatrix} p_{1,1} & \cdots & p_{1,d} & \cdots & p_{1,m} \\ \vdots & \ddots & \vdots & \iddots & \vdots \\ p_{x,1} & \cdots & p_{x,d} & \cdots & p_{x,m} \\ \vdots & \iddots & \vdots & \ddots & \vdots \\ p_{N,1} & \cdots & p_{N,d} & \cdots & p_{N,m} \end{bmatrix}_{N \times 1} \tag{45}$$

$$p_{x,d} = l + r.(u_d - l_d) \tag{46}$$

here *P*signifies that the population initialization matrix, $P_x$denoted as i^{th}PO number, $p_{x,d}$defines its d^{th}dimension in the search area and *N*denotes the number of population members. *m*denotes the number of decision variables, *r*denotes chance numbers between $[0,1]$, $l_d$and $u_d$are the lower and upper limits of d^{th}decision variables, correspondingly. The objective function for the PO algorithm is expressed as,

$$Fit = \begin{bmatrix} Fit_1 \\ \vdots \\ Fit_x \\ \vdots \\ Fit_N \end{bmatrix}_{N \times 1} = \begin{bmatrix} Fit(P_1) \\ \vdots \\ Fit(P_i) \\ \vdots \\ Fit(P_N) \end{bmatrix}_{N \times 1} \tag{47}$$

here *Fit* denotes the evaluated objective function vector, Fit_xdenotes the estimated objective function by using x^{th} PO member. The process of updating the position of the PO algorithm population involves two distinct stages: (i) exploration, which entails a predator assaulting the pufferfish, and (ii) exploitation, which involves replicating the pufferfish's defense mechanism against the predator.

3.5.1.2. Exploration phase. Each member of the population has a set of pufferfish that they are identified using,

$$PC_x = \{P_z : Fit_z < Fit_x and z \neq x\} \tag{48}$$

where $x = 1,2,\ldots\ldots\ldots\ldots,N$and $z \in \{1,2,\ldots\ldots\ldots,N\}$. PC_x denotes population member with enhanced fitness value than x^{th}predator, and $Fit_z$is its fitness function value. Every PO member's new problem-solving space position is calculated using modeling of the predator's approach to pufferfish. It is defined as,

$$p^{s1}_{x,y} = p_{x,y} + r_{x,y}.\left(PS_{x,y} - X_{x,y}.p_{x,y}\right) \tag{49}$$

The relevant member's old position is replaced by the new one, and the new one improves the fitness value. It is expressed as,

$$P_x = \begin{cases} P^{s1}_x, Fit^{s1}_x \leq Fit_x; \\ P_x, else \end{cases} \tag{50}$$

where PS_xdenoted as nominated pufferfish for x^{th}predator carefully chosen randomly form PC_xset, $PS_{x,y}$is its y^{th}dimension, and P^{s1}_xdenotes new location designed for x^{th}predator based on the first phase of the PO algorithm. $p^{s1}_{x,y}$is its y^{th}dimension, $Fit^{s1}_x$is its fitness function value, $r_{x,y}$are random numbers from intervals $[0,1]$and $X_{x,y}$represents the numbers which are randomly chosen as 1 or 2.

3.5.1.3. Exploitation phase. An algorithm that mimics the pufferfish's defense mechanism in the face of predator attacks is used to update the population's position. A new position is calculated for each PO member,

$$p^{s2}_{x,y} = p_{x,y} + \left(1 - 2r_{x,y}\right).\frac{u_y - l_y}{i} \tag{51}$$

The matching member is then replaced by this new location if it raises the value of the objective function in accordance with P_x,

$$P_x = \begin{cases} P^{s2}_x, Fit^{s2}_x \leq Fit_x; \\ P_x, else \end{cases} \tag{52}$$

where, P^{s2}_xdenotes the new location computed for x^{th}predator based on the second phase of the PO and $p^{s2}_{x,y}$ is its y^{th}dimension. $Fit^{s2}_x$is its objective function value, $r_{x,y}$are arbitrary amounts from intervals $[0,1]$, and *i*denotes repetition counter. By updating the positions of every PO member, the first algorithm iteration is finished based on the phase of exploration and exploitation. The program then moves on to the following iteration and continues processing if updating PO members' positions using Eqs. (48) through (51).

4. Results and discussion

This section provides various experimental analyses for proposed and existing approaches using IoT-23 and CICIoT2023 Dataset. System setting details are described in Table 3. Hyperparameter details about the proposed approach are given in Table 4.

4.1. Performance measure

Mathematical expression for various performance metrics is described in this section.

4.1.1. Accuracy

Accuracy is the parameter that is utilized in many models to evaluate class performance. It is computed by dividing the total number of forecasts by the number of predictions that come true.

$$ACC = \frac{TP_s + TN_s}{TP_s + TN_s + FP_s + FN_s} \tag{53}$$

4.1.2. F1 score

The F1 Score evaluates the trade-offs between precision and recall while taking FP and FN into account. It is a measure of an ML model's overall accuracy and has an expression in the equation below;

$$F_1 Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (54)$$

4.1.3. FAR

The false alarm rate (FAR), or the number of false alerts per total number of warnings or alarms, can be computed using the formula below,

$$FAR = \frac{FP_s}{FP_s + TN_s} \quad (55)$$

4.1.4. DR

The DR parameter is utilized to evaluate performance using the fault detection method.

$$DR = \frac{TP_s}{TP_s + FN_s} \quad (56)$$

4.1.5. RMSE

The root mean square error (RMSE) is calculated by taking the square root of the average of the squared differences between the anticipated and actual values.

$$RMSE = \sqrt{\frac{\sum_{i=1}^{N})^2}{N}} \quad (57)$$

4.1.6. Mean square error (MAE)

The Mean Absolute Error (MAE) quantifies the average size of errors in a given set of forecasts, disregarding the direction of the errors. It is expressed as,

$$MAE = \frac{\Sigma|actual - prediction|}{numberofobservations} \quad (58)$$

4.2. Performance evaluation

The performance evaluation of the proposed approach based on the IoT-23 and CICIoT2023 datasets is described in this section.

4.2.1. Results for 80/20 train test partitions

This section's results are taken for an 80:20 training and testing ratio.

4.2.1.1. Evaluation of IoT-23 dataset. Numerous performance metrics such as accuracy, F1 score, FAR, DR, RMSE, and MAE are evaluated for other methods such as RF, SVM, AE, and K means clustering with autoencoder (K+AE) and proposed approach using IoT-23 dataset. Fig. 2 shows the performance evaluation of accuracy, F1 score, and DR metrics.

Here, the proposed and existing approaches are compared in terms of accuracy, F1 score, and DR. Compared to other models, the proposed approach produces high-performance results. Existing models produce less performance due to issues like overfitting, which takes more time for computation. The accuracy performance for the proposed approach is 98.63 %, 98.02 % F1 score, and 98.77 % DR. The proposed approach produces high performance due to reducing issues like overfitting and complexity. By using an ECO algorithm, the proposed approaches solve the overfitting and complexity issues. FAR performance comparison is described in Fig. 3.

Table 3
System configuration specifications.

Device specification	details
Processor	Intel(R) Core(TM) i5–3470 CPU @ 3.10 GHz 3.10 GHz
Installed RAM	16.0 GB
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display.

Table 4
Hyperparameter details.

Sl. No	Hyper-parameters	Particulars
1.	epochs	300
2.	Initial learning rate	0.001
3.	Batch size	32
4.	Optimizer	Adam
5.	Activation function	ReLU, Sigmoid
6.	Drop out factor	0.2
7.	Iterations	100
8.	Loss	Sparse categorical cross entropy
9.	Encoder layer	6
10.	Decoder layer	8
11.	Training testing ratio	80/20 and 70/30

Here, the FAR performance comparison for proposed and existing approaches is described. The existing model can obtain higher values when analyzing error rates using the FAR parameter. This parameter can be used to analyze the error rate of the suggested method, but it can yield fewer results than other systems. FAR performance for the proposed approach is 0.0123, while other approaches produce more than 0.01 FAR performance results. By using the ZdAD-UML model, the proposed approach produces less FAR performance. A comparison of the RMSE and MAE performance is shown in Fig. 4.

Here, the polar plot is described for a proposed and existing approach to compare the RMSE and MAE performance. The Proposed approach obtained 0.1172 and 0.0137 RMSE and MAE performance using the IoT-23 dataset. The range between each circle in a polar plot is 0.05; overall, the plotted graph considered 6 circles. Therefore, a lower RMSE and MAE performance describes that the proposed approach predicts more accurate detection. The ROC curve for the proposed approach using the IoT-23 dataset is shown in Fig. 5.

The proposed approach ROC curve is plotted between the false positive rate (FPR) and the True positive rate (TPR). The area under the ROC curve, which is shown in the graph, is used to calculate AUC. The graph shows that the AUC performance for the proposed approach is 0.956 in zero-day attack detection using the IoT-23 dataset. This analysis shows that the proposed approach maintains balance on data samples in attack detection.

4.2.1.2. Evaluation of the CICIoT2023 dataset. Proposed and other models such as RF, SVM, AE, and K+AE are compared for various performance metrics in terms of accuracy, F1 score, FAR, DR, RMSE, and MAE using the CICIoT2023 dataset to detect the zero-day attack. Accuracy, F1 score, and DR performance comparison are given in Fig. 6.

Here, Accuracy, F1 score, and DR performance are calculated for

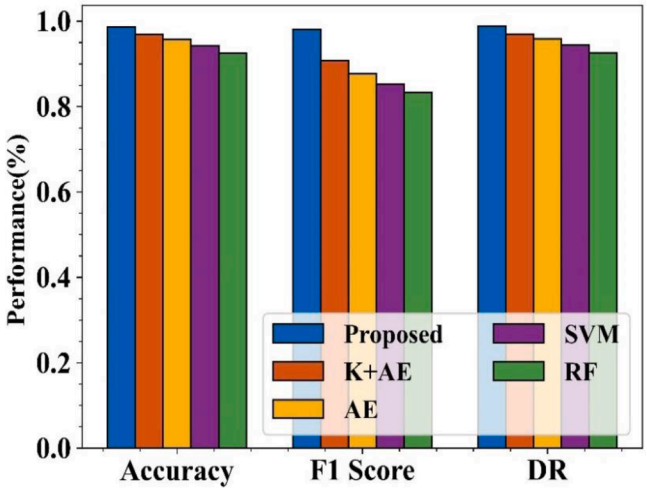


Fig. 2. Performance comparison of accuracy, F1 score, and DR.

proposed and existing approaches by using the CICIoT2023 dataset. Accuracy, F1 score, and DR values for the proposed approach using the CICIoT2023 dataset are 98.51 %, 98.5 %, and 98.04 %, respectively. The performance of current methods is reduced because their parameters have not been adjusted. The proposed technique yields somewhat different values than current systems, as shown in the comparison graphs. A comparison of FAR performance is given in Fig. 7.

FAR performance comparison using the CICIoT2023 dataset for proposed and existing methods is presented in Fig. 7. The proposed approach produces 0.0196 FAR performance in zero-day attack detection using the CICIoT2023 dataset. This parameter can be used to analyze the error rate of the proposed method and yield fewer results than other systems. Polar plot analysis for RMSE and MAE performance is shown in Fig. 8.

A polar plot is taken to show the RMSE and MAE performance of proposed and existing approaches using the CICIoT2023 dataset in zero-day attack detection. The presented polar graph includes 6 circles, and the range of each circle is 0.05. The RMSE and MAE performance for the proposed approach is 0.3057 and 0.0935, respectively, using the CICIoT2023 dataset. Compared to other models, the proposed approach obtains less RMSE and MAE value. The low performance of RMSE and MAE indicates that the proposed approach can effectively perform zero-day attack detection using the CICIoT2023 dataset. ROC curve for the proposed approach using the CICIoT2023 dataset is shown in Fig. 9.

This analysis shows that the proposed approach maintains balance on data samples in attack detection. The area under the ROC curve, which is shown in the graph, is used to calculate AUC. The graph shows that the AUC performance for the proposed approach is 0.985 in zero-day attack detection using the IoT-23 dataset.

4.2.2. *Results for 30–70 test-train partitions*

Results are evaluated for 30–70 Test-Train partitions in this section for both proposed and baseline models.

4.2.2.1. Evaluation results for IoT-23 dataset. Table 5 shows the performance comparison of the proposed and other models using the IoT-23 dataset. Here, the proposed approach obtains efficient results in terms of accuracy, F1-score, detection rate, False Alarm Rate, Zero-data Detection Rate, RMSE, and MAE compared to other methods.

The Proposed model proves superior in detecting zero-day network intrusion attacks when compared to four commonly used models, including K+AE, AE, SVM, and RF. Compared to the Proposed model scores, K+AE delivers a decreased accuracy of 0.9392 coupled with detection rates of 0.9398. The hybrid method proves effective in basic anomaly identification. Yet, its performance remains inferior to that of the proposed model because it lacks the ability to process zero-day

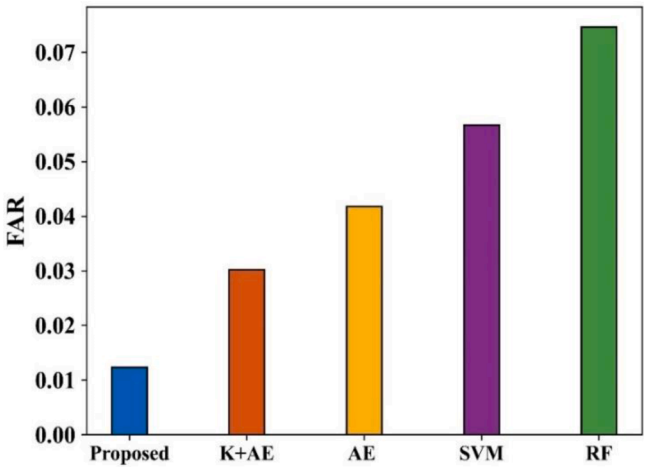


Fig. 3. FAR performance comparison.

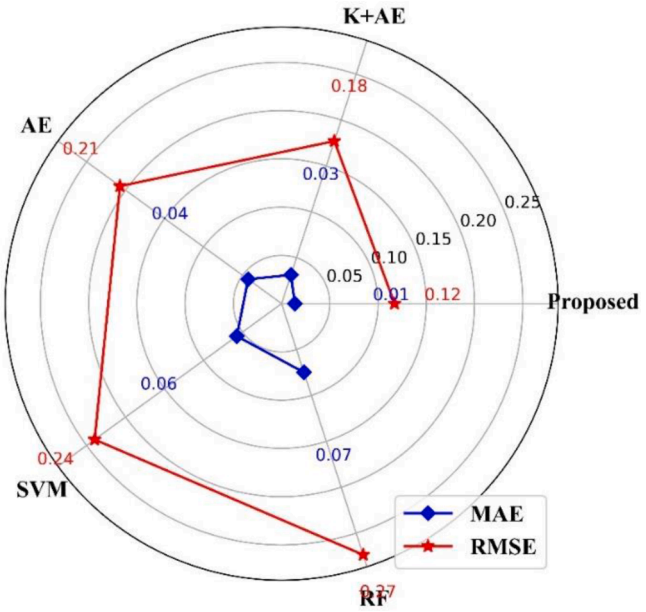


Fig. 4. Polar plot for RMSE and MAE performance.

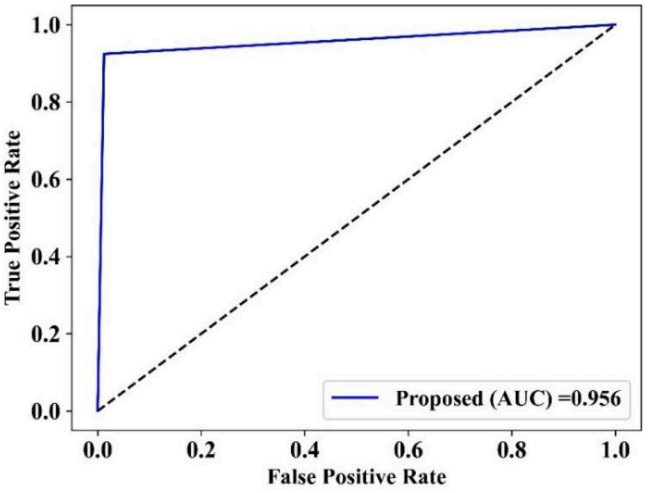


Fig. 5. ROC curve for a proposed approach using the IoT-23 dataset.

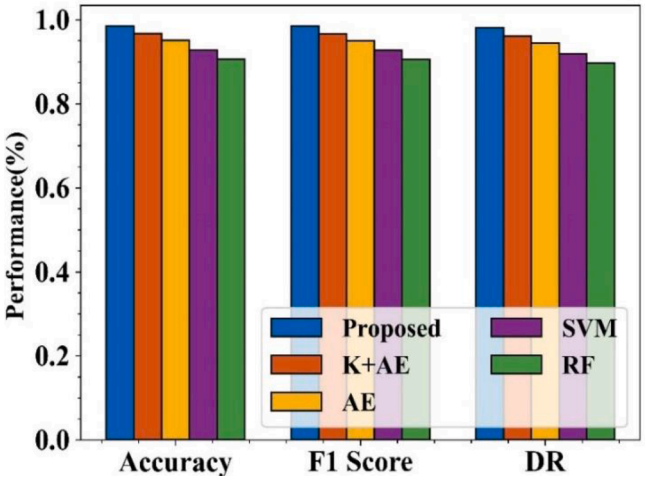


Fig. 6. Accuracy, F1 score, and DR performance comparison.

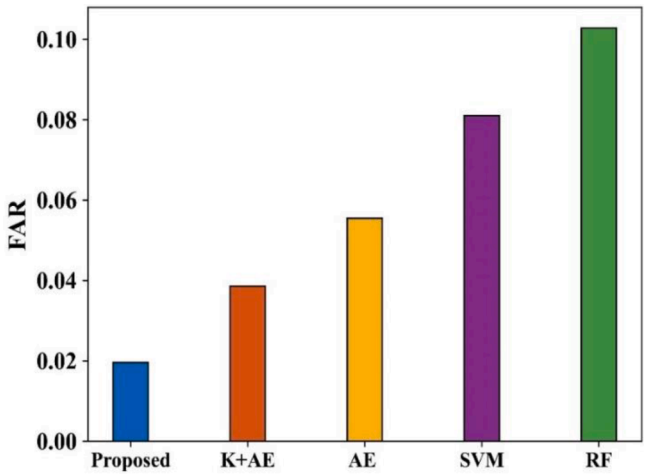


Fig. 7. FAR performance comparison.

attacks and its extended feature extraction and optimization procedures. The AE model demonstrates lower detection performance for anomaly purposes with accuracy at 0.923 and rates at 0.9232, while the proposed model leads the results with 0.9873 and 0.9884, respectively. When implementing the proposed model in real-world applications, its false alarm rate becomes substantially lower than the AE model rate at 0.0216 compared to 0.0768, thereby increasing system reliability. The SVM classical machine learning model faces inferior performance compared to the proposed model in all aspects as the proposed model demonstrates superior accuracy of 0.9873, a detection rate of 0.9884 and a lower false alarm rate of 0.0216. The proposed model shows superiority over SVM because it addresses nonlinear zero-day attacks using its unsupervised learning method. The proposed model outperforms RF regarding all assessment criteria since it achieves higher accuracy, 0.9873, and a better F1-score of 0.9833, and it shows a lower false alarm rate of 0.0216. The Proposed model uses advanced unsupervised learning methods and optimization algorithms (HOA and POA), which provide superior attack pattern detection of complex and previously unknown attack scenarios over standard models, including SVM and RF or hybrid methods K-Means + Auto Encoder.

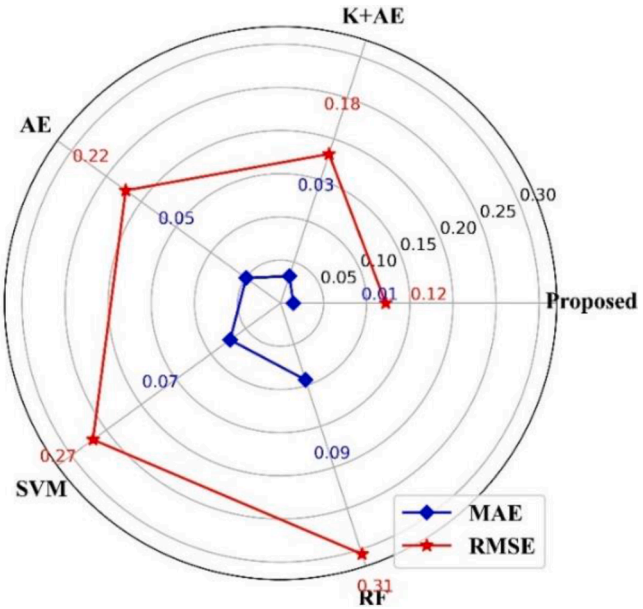


Fig. 8. Polar plot for RMSE and MAE performance.

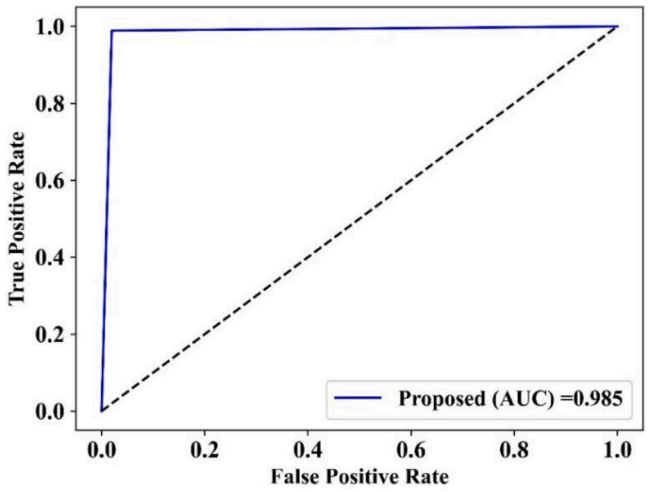


Fig. 9. ROC analysis of the proposed approach using the CICIoT2023 dataset.

4.2.2.2. Evaluation results for the CICIoT2023 dataset. Table 6 shows a performance comparison of the proposed and other models using the CICIoT2023 dataset. Here, the proposed technique compares with some baseline models, namely K+AE, AE, SVM, and RF, due to their varied approaches in anomaly detection and their relevance in Zero-day attack detection on IoT. K+AE integrates clustering and dimensionality reduction to detect anomalies, but the proposed technique outperforms it with advanced feature extraction through EKC-WCAE and optimization algorithms (POA and HOA). The proposed feature extraction and optimization algorithms allow the proposed technique to handle complex attack patterns better, as seen in its higher accuracy and lower false alarm rate. The proposed approach uses optimization algorithms for feature selection and threshold calculation, helping in data redundancy, minimizing computational complexity, and lowering false alarm rates. The Proposed model's ability to generalize from unlabeled data and adapt to evolving attack types makes it more effective, as reflected in its better performance across all metrics, including accuracy, detection rate, and error metrics.

4.2.3. Performance of proposed approach in terms of including with and without feature extraction

Experimental results in terms of with and without feature extraction of the proposed approach are given in Table 7.

Feature extraction typically improves the quality of the data fed into the model, which can lead to better performance. By extracting the most critical features, the model can focus on the aspects of the data that are most indicative of anomalies or zero-day attacks, thus improving detection accuracy. Feature extraction typically improves the quality of the data fed into the model, which can lead to better performance. By extracting the most critical features, the model can focus on the aspects of the data that are most indicative of anomalies or zero-day attacks, thus improving detection accuracy. Without feature extraction, the system would have to deal with noisy, redundant, and less informative

Table 5
Performance results for IoT-23 dataset.

Metric	Proposed	K + AE	AE	SVM	RF
Accuracy	**0.9873**	0.9392	0.923	0.9082	0.8978
F1-score	**0.9833**	0.8481	0.8302	0.8178	0.8101
Detection Rate	**0.9884**	0.9398	0.9232	0.908	0.8973
False Alarm Rate	**0.0216**	0.0602	0.0768	0.092	0.1027
Zero-data Detection Rate	**98.9271**	98.1458	98.4062	98.6667	98.6667
RMSE	**0.1809**	0.2466	0.2775	0.3029	0.3198
MAE	**0.0227**	0.0608	0.077	0.0918	0.1022

Table 6
Performance results for the CICIoT2023 dataset.

Metric	Proposed	K+AE	AE	SVM	RF
Accuracy	**0.9893**	0.9467	0.9226	0.8969	0.8929
F1-score	**0.9895**	0.9463	0.922	0.8962	0.892
Detection Rate	**0.9893**	0.9402	0.9135	0.8934	0.8782
False Alarm Rate	**0.0197**	0.0598	0.0865	0.1066	0.1218
Zero-data Detection Rate	**98.9462**	95.2142	93.0264	89.9727	90.5196
RMSE	**0.1863**	0.2309	0.2781	0.3212	0.3273
MAE	**0.0147**	0.0533	0.0774	0.1031	0.1071

Table 7
Experimental results in terms of with and without feature extraction of the proposed approach.

Metrics	Train/Test Split	With Feature Extraction (EKC-WCAE)		Without Feature Extraction	
		IoT-23	CICIoT2023	IoT-23	CICIoT2023
Accuracy (%)	80/20	98.65	98.51	95.19	95.54
	70/30	98.73	98.93	95.92	94.76
Detection rate (%)	80/20	98.77	98.04	94.64	92.86
	70/30	98.84	98.95	93.85	93.78
F1-score (%)	80/20	98.02	98.5	93.97	93.68
	70/30	98.33	98.95	95.78	92.86
False alarm rate	80/20	0.0123	0.0196	0.521	0.493
	70/30	0.0216	0.0197	0.489	0.482

Table 8
Existing performance comparison.

Methods	Accuracy (%)
Decision tree [24]	95.23
GRP+K-means [24]	92.02
GRP+SGDOneclass [24]	90.53
GRP+GMM [24]	89.10
Ensemble classifier [24]	94.50
ANN [25]	96.73
RNN [25]	96.4
PSO-SVM [25]	97.09
KNN [25]	96.3
DL-Droid [45]	97.8
FAMD [46]	97.40
PermDroid [47]	98
MAPAS [48]	91.27
MLDroid [49]	98
DroidMalwareDetector [50]	90
Proposed	98.69 and 98.72 for IoT-23 and CICIoT2023 dataset

data. This can make it harder to identify meaningful patterns or anomalies, leading to lower accuracy in detecting zero-day attacks. Using raw features could increase the time and resources required for training the model, as more features need to be processed. This can also slow down the detection process, which is critical in an IoT security setting where real-time detection is essential.

4.3. Discussion

The primary goal of this research work is to use the ZdAD-UML model to detect zero-day attacks and improve the cybersecurity of IoT networks. The Major novelty of the proposed approach relies on the EKC-WCAE model. The role of k means clustering and the AE model is to produce effective performance, which leads to accurate zero-day attack detection. Moreover, using an ECO algorithm for weight optimization, the hyperparameter of the model is tuned to enhance performance. Finally, the anomaly score is computed and contrasted with the threshold value, and the proposed approach uses POA to perform the threshold calculation. Compared to the existing approaches, the proposed approach produces effective performance results in zero-day attack detection by using introduced novel approaches. More existing model comparisons for zero-day detection are shown in Table 8.

Here, the proposed approach is compared with existing models for analyzing accuracy performance. Accuracy performance attained by the proposed approach is 98.51 % and 98.63 % using the IoT-23 and CICIoT2023 datasets. Meanwhile, all other models produce less accuracy performance compared to the proposed approach. The time complexity comparison is described in Table 9.

Here, the time complexity analysis for proposed and other approaches in zero-day attack detection using the IoT-23 and CICIoT2023 datasets is summarized. Compared to other models, the proposed approach takes less time to detect zero-day attacks, that is, 1.99 and 1.87 ms, respectively. A performance comparison of proposed and state-of-the-art methods using the IoT-23 dataset is given in Table 10. A performance comparison of proposed and state-of-the-art methods using the CICIoT 2023 dataset is described in Table 11.

Table 9
Time complexity comparisons.

Methods	Dataset	Time complexity (ms)
RF	IoT-23	3.24
	CICIoT2023	3.45
SVM	IoT-23	3.12
	CICIoT2023	3.09
AE	IoT-23	3.02
	CICIoT2023	2.98
K+AE	IoT-23	2.85
	CICIoT2023	2.42
Proposed	**IoT-23**	**1.99**
	CICIoT2023	**1.87**

Table 10
Performance comparison of proposed and state-of-the-art methods using the IoT-23 dataset.

Methods	Accuracy (%)
Bayesian hyperparameter optimization [28]	96.07
GRU-LSTM and GRU-Softmax [29]	96 and 76
BGO-SVM [30]	97
Proposed	**98.69**

Table 11
Performance comparison of proposed and state-of-the-art methods using CICIoT 2023 dataset.

Methods	Accuracy (%)
CNN, DNN, RNN, LSTM [31]	91.27
deepCLG [32]	98
CNN with LSTM [33]	92
CNN with LSTM [34]	98.42
Proposed	**98.72**

5. Conclusion

An intelligent ZdAD-UML model is presented in this research paper to identify zero-day attacks and enhance the cybersecurity of IoT networks. Data cleaning, Min-max Normalization, and One-hot encoding techniques are used in the pre-processing stage to make input data for further analysis. HOA is used in the feature selection process to choose ideal features, minimize data redundancy, and reduce complexity. EKC-WCAE model is used to remove the features; after training the AE, the latent representations are clustered using the K-means clustering algorithm to organize the identical traffic patterns into clusters. The evaluation of the reconstruction error is done between the original and the reconstructed data. Then, the anomaly score is computed and equated

with a threshold value to identify attacks; here, POA is used to calculate the threshold value. Consequently, the suggested EKC-WCAE-based unsupervised modeling offers better performance in identifying and mitigating zero-day DDoS attacks in IoT networks and enhancing security against emerging threads. For the IoT-23 and CICIoT2023 datasets, the proposed approach produces 98.51 % and 98.63 % accuracy performance in the 80–20 train test ratio, respectively. The proposed technique attains an accuracy of 98.73 % and 98.93 % using IoT-23 and CICIoT2023 datasets for a 70–30 train test ratio. One limitation of the proposed model is that it depends on optimization algorithms, which leads to increased processing time in large-scale IoT networks. Future work will focus on optimizing the efficiency of these algorithms, possibly by exploring alternative, more computationally efficient optimization algorithms that maintain high detection accuracy while reducing resource consumption. Another limitation is reliance on datasets like the IoT-23 and CICIoT2023 datasets, which didn't show the diversity of real-world IoT network traffic and attack vectors. Further research will involve including additional, more varied datasets to further improve the model's generalizability and robustness against new types of attacks. Its performance on extremely rare or sophisticated attacks could be further refined. Investigating hybrid models that combine the strengths of both supervised and unsupervised learning could be a promising avenue for improving detection capabilities. Lastly, real-time deployment and testing in dynamic, large-scale IoT environments would be essential for assessing the model's practical applicability and scalability. By addressing these limitations and exploring these improvements, future iterations of the proposed model could achieve even greater accuracy, efficiency, and adaptability in safeguarding IoT networks against evolving cybersecurity threats.

Funding information

No funding is provided for the preparation of the manuscript.

CRediT authorship contribution statement

Apoorva Jain: Writing – original draft, Conceptualization. **Renu Bagoria:** Writing – review & editing, Investigation. **Praveen Arora:** Visualization, Validation, Methodology.

Declaration of competing interest

No conflict of Interest.

Data availability

No data was used for the research described in the article.

References

[1] S. Criollo-C, A. Guerrero-Arias, D. Buenaño-Fernández, S. Luján-Mora, Usability and workload evaluation of a cybersecurity educational game application: a case study, IEEe Access. 12 (2024) 12771–12784.
[2] M. Khan, L. Ghafoor, Adversarial machine learning in the context of network security: challenges and solutions, J. Comput. Intell. Robot. 4 (1) (2024) 51–63.
[3] M.S. Wani, M. Rademacher, T. Horstmann, M. Kretschmer, Security vulnerabilities in 5G Non-stand-alone networks: a systematic analysis and attack taxonomy, J. Cybersecur. Priv. 4 (1) (2024) 23–40.
[4] N. Tatipatri, S.L. Arun, A comprehensive review on cyber-attacks in power systems: impact analysis, detection, and cyber security, IEEe Access. 12 (2024) 18147–18167.
[5] N. Jeffrey, Q. Tan, J.R. Villar, A hybrid methodology for anomaly detection in cyber–physical systems, Neurocomputing 568 (2024) 127068.
[6] S.H. Mohammed, A. Al-Jumaily, M.S. Jit Singh, V.P. Gil Jiménez, A.S. Jaber, Y. Soubhi Hussein, M.M. Abdul Kader Al-Najjar, D. Al-Jumeily, A review on the evaluation of feature selection using machine learning for cyber-attack detection in smart grid, IEEe Access. 12 (2024) 44023–44042.
[7] P.K. Chaudhary. AI, ML, and large language models in cybersecurity. Preprint (2024).
[8] K. Shaukat, S. Luo, V. Varadharajan, A novel machine learning approach for detecting first-time-appeared malware, Eng. Appl. Artif. Intell. 131 (2024) 107801.
[9] M.H. Reshadi, W. Li, W. Xu, P. Omashor, A. Dinh, S. Dick, Y. She, M. Lipsett, Deep-shallow metaclassifier with synthetic minority oversampling for anomaly detection in a time series, Algorithms 17 (3) (2024) 114.
[10] D. Lightbody, D.M. Ngo, A. Temko, C.C. Murphy, E. Popovici, Dragon_Pi: IoT side-channel power data intrusion detection dataset and unsupervised convolutional autoencoder for intrusion detection, Fut. Internet. 16 (3) (2024) 88.
[11] L. Shen, M. Fang, J. Xu, GHGDroid: global heterogeneous graph-based android malware detection, Comput. Secur. 141 (2024) 103846.
[12] Z. Liu, R. Wang, N. Japkowicz, H.M. Gomes, B. Peng, W. Zhang, SeGDroid: an Android malware detection method based on sensitive function call graph learning, Expert. Syst. Appl. 235 (2024) 121125.
[13] S. Akshaya, G. Padmavathi, Enhancing zero-day attack prediction a hybrid game theory approach with neural networks, Int. J. Intell. Syst. Appl. Eng. 12 (7s) (2024) 643–663.
[14] C. Liang, Clustering analysis of unlabeled data and weak-label detection analysis method integrating Soft computing technology, IEEe Access. 12 (2024) 6852–6863.
[15] S.K. Hegde, P. William, M.S. Basvant, A. Deepak, A. Badhoutiya, A.L.N. Rao, A. Srivastava, A. Shrivastava, Energy-efficient Bio-inspired hybrid deep learning model for network intrusion detection based on intelligent decision making, Int. J. Intell. Syst. Appl. Eng. 12 (16s) (2024) 306–319.
[16] O. Adeniyi, A. Safaa Sadiq, P. Pillai, M. Aljaidi, O. Kaiwartya, Securing mobile edge computing using hybrid deep learning method, Computers 13 (1) (2024) 25.
[17] M. Soltani, B. Ousat, M.J. Siavoshani, A.H. Jahangir, An adaptable deep learning-based intrusion detection system to zero-day attacks, J. Inf. Secur. Appl. 76 (2023) 103516.
[18] O. Jurečková, M. Jureček, M. Stamp, F. Di Troia, R. Lórencz, Classification and online clustering of zero-day malware, J. Comput. Virol. Hacking Tech. (2024) 1–14.
[19] M. Ali, M. Shahroz, M.F. Mushtaq, S. Alfarhood, M. Safran, I. Ashraf, Hybrid machine learning model for efficient botnet attack detection in IoT environment, IEEe Access. 12 (2024) 40682–40699.
[20] S. Sharma, V. Kumar, K. Dutta, Multi-objective optimization algorithms for intrusion detection in IoT networks: a systematic review, Internet Things Cyber-Phys. Syst. 4 (2024) 258–267.
[21] R. Xu, G. Wu, W. Wang, X. Gao, A. He, Z. Zhang, Applying self-supervised learning to network intrusion detection for network flows with graph neural network, Comput. Netw. 248 (2024) 110495.
[22] M.H. Behiry, M. Aly, Cyberattack detection in wireless sensor networks using a hybrid feature reduction technique with AI and machine learning methods, J. Big. Data 11 (1) (2024) 16.
[23] S. SakthiMurugan, S. Kumaar, V. Vignesh, P. Santhi, Assessment of zero-day vulnerability using machine learning approach, EAI Endorsed Trans. Internet Things 10 (2024).
[24] M. Roopak, S. Parkinson, G.Y. Tian, et al., An unsupervised approach for the detection of zeroday ddos attacks in iot networks, Authorea (2024).
[25] N. Omer, A.H. Samak, A.I. Taloba, A. El-Aziz, M. Rasha, Cybersecurity threats detection using optimized machine learning frameworks, Comput. Syst. Sci. Eng. 48 (1) (2024).
[26] A.P. Ekong, A. Etuk, S. Inyang, M. Ekere-obong, Securing against zero-day attacks: a machine learning approach for classification and organizations' Perception of its impact, J. Inf. Syst. Inform. 5 (3) (2023) 1123–1140.
[27] N. Alam, and M. Ahmed. Zero-day network intrusion detection using machine Learning approach. no. April (2023) 194–201.
[28] N. Abbas, and F. Al Mukhaini. Enhancing intrusion detection in IoT environment using decision trees and Bayesian hyperparameter optimization.
[29] C. He, Chao, Internet of Things data intrusion detection under GRU-LSTM algorithm, in: Fourth International Conference on Telecommunications, Optics, and Computer Science (TOCS 2023) 13161, SPIE, 2024, pp. 301–307.
[30] D. Kumar, P.P. Pawar, B. Ananthan, S. Rajasekaran, T. Velu Prabhakaran, Optimized support vector machine based fused IOT network security management, in: 2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT), IEEE, 2024, pp. 1–5.
[31] S. Hizal, U. Cavusoglu, D. Akgun, A novel deep learning-based intrusion detection system for IoT DDoS security, Intern. Things 28 (2024) 101336.
[32] Q. Gulzar, K. Mustafa, Enhancing network security in industrial IoT environments: a DeepCLG hybrid learning model for cyberattack detection, Int. J. Mach. Learn. Cybern. (2025) 1–19.
[33] H. Alzahrani, T. Sheltami, A. Barnawi, M. Imam, A. Yaser, A lightweight intrusion detection system using convolutional neural network and long short-term memory in fog computing, Comput. Mater. Amp; Contin. 80 (3) (2024) 4703–4728.
[34] A. Gueriani, H. Kheddar, A.Cherif Mazari, Enhancing IoT security with CNN and LSTM-based intrusion detection systems, in: 2024 6th International Conference on Pattern Analysis and Intelligent Systems (PAIS), IEEE, 2024, pp. 1–7.
[35] N. Abdalgawad, A. Sajun, Y. Kaddoura, I.A. Zualkernan, F. Aloul, Generative deep learning to detect cyberattacks for the IoT-23 dataset, IEEe Access. 10 (2021) 6430–6441.
[36] E.C.P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment, Sensors 23 (13) (2023) 5941.
[37] X. Zheng, L. Chen, C. Yu, Z. Lei, Z. Feng, Z. Wei, Gearbox compound fault diagnosis in edge-IoT based on legendre multiwavelettransform and convolutional neural network, Sensors 23 (21) (2023) 8669.
[38] S. Tangwannawit, P. Tangwannawit, An optimization clustering and classification based on artificial intelligence approach for internet of things in agriculture, IAES Int. J. Artif. Intell. 11 (1) (2022) 201.

[39] D. Reilly, M. Taylor, P. Fergus, C. Chalmers, S. Thompson, The categorical data conundrum: heuristics for classification problems—a case study on domestic fire injuries, IEEe Access. 10 (2022) 70113–70125.
[40] M.H. Amiri, N.M. Hashjin, M. Montazeri, S. Mirjalili, N. Khodadadi, Hippopotamus optimization algorithm: a novel nature-inspired optimization algorithm, Sci. Rep. 14 (1) (2024) 5032.
[41] W. Lu, Improved K-means clustering algorithm for big data mining under Hadoop parallel framework, J. Grid. Comput. 18 (2) (2020) 239–250.
[42] Y. Yang, et al., Network intrusion detection based on supervised adversarial variational auto-encoder with regularization, IEEe Access. 8 (2020) 42169–42184.
[43] H. Jia, H. Rao, C. Wen, S. Mirjalili, Crayfish optimization algorithm, Artif. Intell. Rev. 56 (Suppl 2) (2023) 1919–1979.
[44] M.C. Catalbas, A. Gulten, Pufferfish optimization algorithm: a bioinspired optimizer, Handb. Intell. Comput. Optim. Sustain. Dev. (2022) 461–485.
[45] M.K. Alzaylaee, S.Y. Yerima, S. Sezer, DL-Droid: deep learning based android malware detection using real devices, Comput. Secur. 89 (2020) 101663.
[46] H. Bai, N. Xie, X. Di, Q. Ye, Famd: a fast multifeature android malware detection framework, design, and implementation, IEEe Access. 8 (2020) 194729–194740.
[47] A. Mahindru, H. Arora, A. Kumar, S.K. Gupta, S. Mahajan, S. Kadry, J. Kim, PermDroid a framework developed using proposed feature selection approach and machine learning techniques for Android malware detection, Sci. Rep. 14 (1) (2024) 10724.
[48] J. Kim, Y. Ban, E. Ko, H. Cho, J.H. Yi, MAPAS: a practical deep learning-based android malware detection system, Int. J. Inf. Secur. 21 (4) (2022) 725–738.
[49] A. Mahindru, A.L. Sangal, MLDroid—framework for Android malware detection using machine learning techniques, Neural Comput. Appl. 33 (10) (2021) 5183–5240.
[50] A.T. Kabakus, DroidMalwareDetector: a novel Android malware detection framework based on convolutional neural network, Expert. Syst. Appl. 206 (2022) 117833.